



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Laboratory 2

Components and Connectors

Jeisson Andrés Vergara Vargas

Software Architecture

2026-II

1. Objective

The objective of this lab is to build, deploy and test a set of architectural elements, in order to have a first practical approach to the concepts of **components** and **connectors**.

2. Prerequisites

A computer with a Unix-based OS and [Docker](#) installed.

3. Components

Create a folder **c&c**.

3.1. Component 2

a. Into the **c&c** folder, create a **component-2** folder.

b. Into the **component-2** folder, create an **app** folder.

c. Into the **app** folder, create an **init.py** file (empty).

d. Into the **app** folder, create a **db.py** file:

```
import motor.motor_asyncio
import os

client = None
```

```
db = None

async def init_db():
    global client, db
    client = motor.motor_asyncio.AsyncIOMotorClient(os.environ["DB_HOST"])
    db = client[os.environ["DB_NAME"]]

def get_collection():
    return db["items"]
```

e. Into the **app** folder, create a **schema.py** file:

```
import strawberry
from typing import List
from app.db import get_collection
from bson.objectid import ObjectId

@strawberry.type
class Item:
    id: str
    name: str
    description: str

@strawberry.type
class Query:
    @strawberry.field
    async def items(self) -> List[Item]:
        collection = get_collection()
        results = await collection.find().to_list(100)
        return [Item(id=str(r["_id"]), name=r["name"],
description=r["description"]) for r in results]

@strawberry.type
class Mutation:
    @strawberry.mutation
    async def add_item(self, name: str, description: str) -> Item:
        collection = get_collection()
        result = await collection.insert_one({"name": name, "description":
description})
        return Item(id=str(result.inserted_id), name=name,
description=description)

schema = strawberry.Schema(query=Query, mutation=Mutation)
```

f. Into the **app** folder, create a **main.py** file:

```
import uvicorn
from fastapi import FastAPI
from strawberry.fastapi import GraphQLRouter
```

```

from app.schema import schema
from app.db import init_db

app = FastAPI()

graphql_app = GraphQLRouter(schema)
app.include_router(graphql_app, prefix="/graphql")

@app.on_event("startup")
async def startup_db():
    await init_db()

if __name__ == "__main__":
    uvicorn.run("app.main:app", host="0.0.0.0", port=8000)

```

g. Into the **component-2** folder, create a **requirements.txt** file:

```

fastapi
strawberry-graphql[fastapi]
uvicorn
motor
python-dotenv

```

h. Into the **component-2** folder, create a **Dockerfile** file:

```

FROM python:3.11-slim

WORKDIR /app
ENV PYTHONPATH=/app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

CMD ["python", "app/main.py"]

```

3.2. Component 4

a. Into the **c&c** folder, create a **component-4** folder.

b. Into the **component-4** folder, create an **app** folder.

c. Into the **app** folder, create an **app.py** file:

```

import os
from flask import Flask, jsonify, request

```

```

import mysql.connector

app = Flask(__name__)

def get_db_connection():
    return mysql.connector.connect(
        host=os.environ["DB_HOST"],
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
        database=os.environ["DB_NAME"]
    )

@app.route('/items', methods=['GET'])
def get_items():
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM items")
    items = cursor.fetchall()
    conn.close()
    return jsonify(items)

@app.route('/items', methods=['POST'])
def create_item():
    data = request.json
    name = data.get("name")
    description = data.get("description")

    if not name or not description:
        return jsonify({"error": "Missing name or description"}), 400

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("INSERT INTO items (name, description) VALUES (%s, %s)",
        (name, description))
    conn.commit()
    conn.close()

    return jsonify({"status": "Item created"}), 201

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8001)

```

d. Into the **component-4** folder, create a **requirements.txt** file:

```

flask
mysql-connector-python
python-dotenv

```

e. Into the **component-4** folder, create a **Dockerfile** file:

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

CMD ["python", "app/app.py"]
```

3.3. Components 1 and 3

What are **components 1** and **3**?

4. Deployment

a. Into the **c&c** folder, create a **docker-compose.yml** file:

```
services:

  component-1:
    image: mongo
    restart: always
    container_name: component-1
    ports:
      - "27017:27017"

  component-2:
    build: ./component-2
    container_name: component-2
    ports:
      - "8000:8000"
    environment:
      DB_HOST: mongodb://component-1:27017
      DB_NAME: db
    depends_on:
      - component-1

  component-3:
    image: mysql:8
    container_name: component-3
    environment:
      MYSQL_ROOT_PASSWORD: 123
      MYSQL_DATABASE: db
    ports:
      - "3306:3306"

  component-4:
    build: ./component-4
```

```
container_name: component-4
ports:
  - "8001:8001"
environment:
  DB_HOST: component-3
  DB_USER: root
  DB_PASSWORD: 123
  DB_NAME: db
depends_on:
  - component-3
```

b. Create the Docker images and generate the Docker containers through the execution of the following command:

```
docker-compose up --build
```

5. Connectors

How can we find the connectors?

6. Testing

6.1. Flow 1

a. Enter to the **component-3 container** using a terminal:

```
docker exec -it component-3 sh
```

b. Execute the **MySQL client**:

```
mysql -u root -p
```

Password = 123

c. Select the **db** database:

```
use db;
```

d. Create the **items** table:

```
CREATE TABLE items (
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(255) NOT NULL,  
    description TEXT  
);
```

e. Create an *item* using **component-4** (HTTP request from any HTTP client):

```
curl -X POST http://localhost:8001/items \  
-H "Content-Type: application/json" \  
-d '{"name": "SwArch", "description": "2026-I"}'
```

f. Get all *items* using **component-4** (HTTP request from any HTTP client):

```
curl -X GET http://localhost:8001/items
```

g. Verify *items* into the database:

```
SELECT * FROM items;
```

6.2. Flow 2

a. Open the following resource in a web browser: <http://localhost:8000/graphql>.

b. Create an *item* (using **component-2**):

```
mutation {  
  addItem(name: "SwArch", description: "2026-I") {  
    id  
    name  
    description  
  }  
}
```

c. Get all *item* (using **component-2**):

```
query {  
  items {  
    id  
    name  
    description  
  }  
}
```

d. Execute the **MongoDB client** (for **component-1**):

```
mongosh mongodb://localhost:27017
```

e. Select the **db** database:

```
use db
```

f. Verify *items* into the database:

```
db.items.find().pretty()
```

7. Delivery

7.1. Deliverable

- Component-and-Connector (C&C) View.
- Description of the architectural elements.

7.2. Delivery Deadline

Tuesday, September 29, 2026 (in class).